

Cicli

I cicli possono essere utilizzati per ripetere più volte l'esecuzione di un blocco di istruzioni, fintanto che una certa condizione risulta essere vera. Sono estremamente utili, in quanto consentono di far risparmiare tempo al programmatore, ridurre gli errori e rendere il codice decisamente più leggibile.

While

Sintassi:

```
while(condizione) {  
    //istruzioni  
}
```

Il ciclo while (ciclo con controllo in testa) esegue un certo blocco di istruzioni fintanto che *condizione* risulta essere vera (ha valore *true*).

Esempio:

```
int i = 0;  
while (i < 5) {  
    System.out.println(i);  
    i++;  
}
```

Nell'esempio sopra riportato, viene stampato il valore della variabile *i* fino a quando questa non assume valore 5 (la condizione diventa falsa e quindi il ciclo termina).

Attenzione: per far sì che il ciclo termini e non diventi un loop infinito (esistono casi dove questo comportamento è voluto) è bene essere certi che la condizione prima o poi diventi falsa.

Dal momento che il controllo della condizione viene fatto in testa (all'inizio), il blocco di istruzioni compreso nel ciclo while potrebbero non essere mai eseguite se la condizione è falsa in partenza.

Do/while

Sintassi:

```
do {  
    //istruzioni  
}while (condizione);
```

Anche il ciclo `do/while` esegue un certo blocco di istruzioni fintanto che *condizione* risulta essere vera. La differenza con il ciclo `while` sta nel momento in cui la condizione viene verificata: questo è un ciclo con controllo in coda (alla fine), quindi si è certi che almeno una volta l'intero blocco di istruzioni contenuto nel ciclo venga eseguito, anche se la condizione è falsa in partenza.

Osserviamo la presenza del `;` posto dopo il controllo della condizione.

Esempio:

```
int i = 0;
do {
    System.out.println(i);
    i++;
}while (i < 5);
```

Questo esempio ha lo stesso comportamento di quello precedente.

For

Sintassi:

```
for(espressione1; espressione2; espressione3){
    //istruzioni
}
```

dove:

- *espressione1* viene eseguita una sola volta prima dell'esecuzione del blocco di istruzioni del ciclo;
- *espressione2* definisce la condizione di esecuzione del blocco (il blocco di istruzioni si ripete fintanto che la condizione è vera);
- *espressione3* viene eseguita ogni volta alla fine del blocco di istruzioni del ciclo.

Tipicamente questi cicli si usano quando è noto a priori il numero di volte che il blocco di istruzioni deve essere ripetuto, usando una variabile come contatore delle iterazioni, ma è possibile prevedere una condizione più "complessa", utilizzando operatori logici.

Esempio:

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

Nell'esempio si stampano i numeri da 0 a 4 (come negli esempi precedenti); osserviamo come non ci sia la necessità di incrementare il valore della variabile *i*, in quanto viene specificato direttamente all'interno delle espressioni che compongono il ciclo.

In questo caso:

- la variabile *i* funge da contatore delle iterazioni del ciclo;
- *espressione1* (`int i = 0`) ci indica il valore iniziale del contatore;
- *espressione2* (`i < 5`) ci indica la condizione del ciclo, che si ripete fintanto che la variabile ha valore inferiore a 5;
- *espressione3* (`i++`) è l'aggiornamento della variabile del ciclo (incremento in questo caso).